TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
Khoa Công nghệ và Kỹ thuật

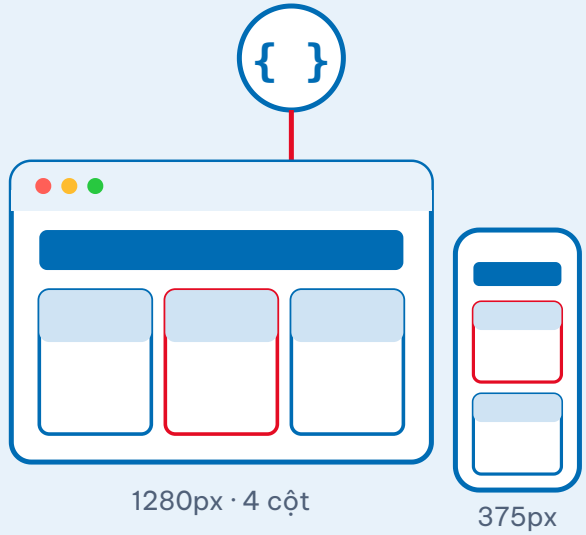
IN4529 · CHƯƠNG 2

CSS hiện đại & Responsive

Lập trình Front End

Giảng viên **ThS. Nguyễn Tuấn Kiệt**
Năm học 2026–2027

CSS · Flexbox · Grid · Responsive

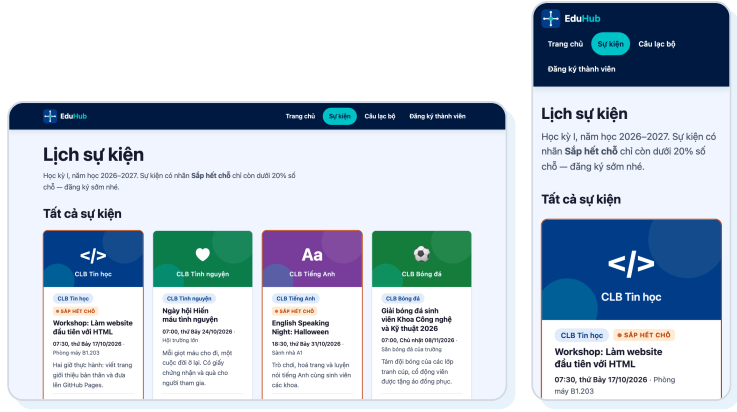



```
.card-grid {
display: grid;
grid-template-columns:
repeat(auto-fit, …);
}
```

0836 980 284 nguyentuankiet@dthu.edu.vn

Nội dung chương 2

- ↺ Ôn lại Chương 1
- **2.1** Selector, specificity, cascade, `@layer`
- **2.2** Box model, đơn vị, màu, font
- **2.3** Flexbox — dàn hàng một chiều
- **2.4** Grid — lưới hai chiều
- **2.5** Responsive mobile-first **MỞ RỘNG**
- **2.6** CSS hiện đại: token, nesting, `:has()`
- **2.7** Tổ chức CSS: BEM · Tailwind v4



Hình 2.1. EduHub Web sau Chương 2 ở 1280px và 375px (ảnh chụp solution).

Giữ nguyên HTML của Ch1, chỉ viết CSS.

Ôn lại Chương 1

Trả lời nhanh trong đầu, không mở slide cũ.

CÂU 1 Trình duyệt dựng trang từ HTML ra **DOM**. CSS thì được dựng thành cái gì, và hai thứ đó gặp nhau ở bước nào?

CÂU 2 Trang `events.html` của bạn có `<nav>`, `<main>`, `aria-current="page"`. Ba thứ này giúp gì cho người dùng trình đọc màn hình?

CÂU 3 Vì sao ta tách **nội dung** (HTML) khỏi **trình bày** (CSS)? Nếu muốn đổi màu xanh của cả 4 trang, sửa ở đâu là nhanh nhất?

Câu 3 chính là lý do chương này có **design token** (2.6).

2.1

PHẦN 1 / 7

PHẦN 1

Selector, specificity, cascade & kế thừa

- Cú pháp rule và các loại selector
- Specificity — điểm cụ thể
- Cascade và kế thừa
- `@layer` — xếp CSS thành tầng

2.1 Ví dụ đời thường: nội quy nhiều cấp

🏫 Nội quy trường

"Sinh viên mặc áo trắng." Áp cho **mọi người** — giống selector thẻ `p`.

👥 Quy định lớp

"Lớp IN4529 mặc áo xanh ngày thứ Hai." Cụ thể hơn — giống `.class`.

🙋 Riêng một bạn

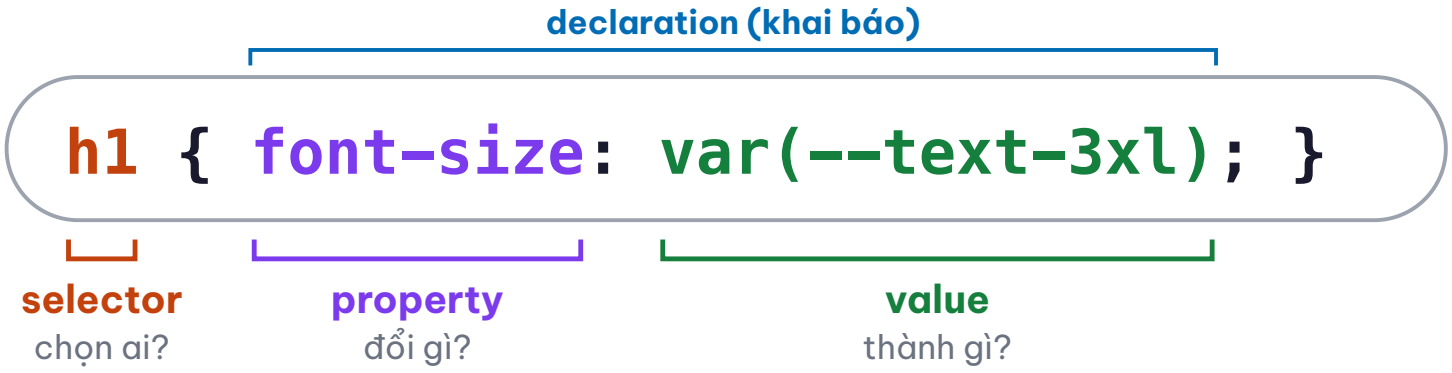
"Bạn An làm MC, mặc áo dài." Cụ thể nhất — giống `#id`.

👨‍👩‍👧 Con giống cha mẹ

Họ, quê quán truyền từ cha mẹ sang con — giống **kế thừa** (màu chữ, font).

ACTIVE RECALL Ba quy định trên cùng nói về áo của bạn An. Áp quy định nào? Vì sao?

2.1 Cú pháp một rule CSS



Cả khối selector { … } gọi là một rule (quy tắc). Một tệp CSS = nhiều rule.

Hình 2.2. Các phần của một rule CSS (tự vẽ).

```
<link rel="stylesheet" href="css/main.css">
```

Gắn CSS bằng `<link>` trong `<head>`. 4 trang chỉ liên kết **một** tệp `main.css`.

Đoạn mã 2.1. Gắn tệp CSS vào trang. Nguồn: `02_Projects/ch02/solution/index.html`, `css/base.css`

2.1 Các loại selector hay dùng

Bảng 2.1. Các loại selector, ví dụ lấy từ CSS của EduHub

LOẠI	VÍ DỤ TRONG EDUHUB	CHỌN PHẦN TỬ NÀO
Thẻ	`h1`, `a`, `img`	Mọi thẻ cùng tên
Class	`.btn`, `.card__title`	Phần tử có `class="btn"`
Thuộc tính	`[aria-current="page"]`	Phần tử có thuộc tính đó
Con cháu	`.card__title a`	`a` nằm *bên trong* `.card__title`
Pseudo-class	`:hover`, `:focus-visible`, `:user-invalid`	Theo **trạng thái** (đang rê chuột, đang focus…)
Pseudo-element	`::before`, `::after`, `::marker`	Một **phần** của phần tử (dấu đầu dòng…)
ID	`#noi-dung`	Một phần tử duy nhất — **không dùng để tạo kiểu**

Các selector lấy từ `02_Projects/ch02/solution/css/*.css`; ID chỉ dùng cho link neo “Bỏ qua…”.

2.1 Specificity — "điểm cụ thể" của selector

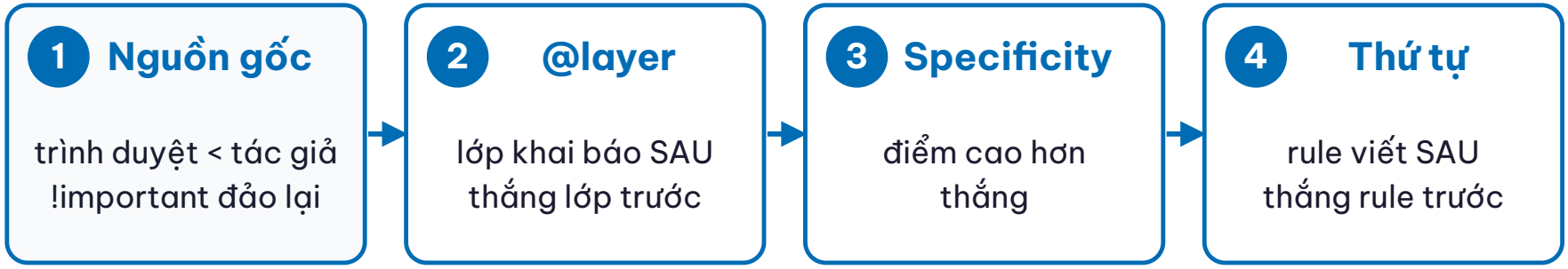
Selector	ID	Class	Thẻ
`h1`	0	0	1
`.card__title a`	0	1	1
`.site-nav__link[aria-current="page"]`	0	2	0
`#noi-dung`	1	0	0
`:where(h1, p)`	0	0	0

So từ cột trái sang phải, giống so số 3 chữ số: 1-0-0 thắng 0-9-9.

Hình 2.3. Bảng điểm specificity của một số selector (tự vẽ).

Hai rule cùng chọn một phần tử, cùng đổi một thuộc tính → rule có **điểm cao hơn** thắng.

2.1 Cascade — trình duyệt chọn rule nào?



Bước trên còn hoà → mới xét bước dưới. Hầu hết "tranh chấp" trong bài tập phân xử ở bước 2–4.

```css
/* base.css (lớp base) */
a { color: var(--color-link); }
/* điểm 0-0-1 */
```

```css
/* components.css */
.site-nav__link { color: … }
/* lớp sau + điểm 0-1-0 → THẮNG */
```

Hình 2.4. Bốn bước phân xử của cascade (tự vẽ).

Cascade (thác nước) = luật phân xử khi nhiều rule cùng nhắm một thuộc tính.

Kế thừa (inheritance)

Kế thừa từ cha

- `color`, `font-family`, `font-size`, `line-height`
- → đặt một lần ở `body`, cả trang dùng theo

Không kế thừa

- `margin`, `padding`, `border`, `background`, `width`
- → phải đặt riêng cho từng phần tử

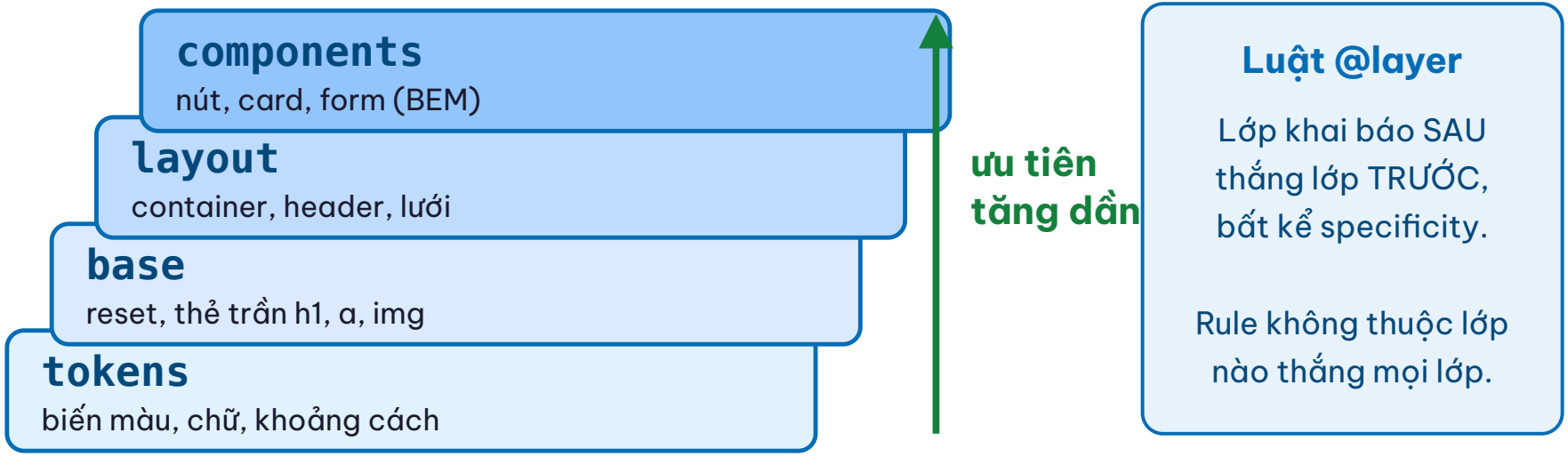
```
body {
  font-family: var(--font-sans);
  font-size: var(--text-base);
  color: var(--color-text);
  …
}

:where(input, button, select, textarea) {
  font: inherit;
  color: inherit;
}
```

Đoạn mã 2.2. Đặt kiểu chữ một lần ở `body`; ô nhập kế thừa font. Nguồn: `02_Projects/ch02/solution/css/base.css` · đã lược chú thích

Ô nhập, nút *mặc định không kế thừa* font → ép bằng giá trị `inherit`.

2.1 @layer — xếp CSS thành các tầng



Hình 2.5. Bốn lớp tokens, base, layout, components xếp chồng theo thứ tự ưu tiên (tự vẽ).

Nhờ `@layer`, không cần "đua điểm" specificity hay `!important`: `components` luôn thắng `base`.

@layer trong EduHub: `main.css`

```css
@layer tokens, base, layout, components;

/* @import phải đứng đầu tệp (chỉ sau dòng @layer ở trên) */
@import url("tokens.css") layer(tokens);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components.css") layer(components);
```

Đoạn mã 2.3. Khai báo thứ tự lớp và nạp 4 tệp CSS. Nguồn: `02_Projects/ch02/solution/css/main.css`

- Dòng 1 **đặt tên và thứ tự** các lớp — đọc một dòng là biết ai thắng ai.
- `@import … layer(…)` nạp từng tệp vào đúng lớp; `@import` phải đứng đầu tệp.
- HTML chỉ cần `<link href="css/main.css">` (ch2-T2).

2.1 Lỗi hay gặp — selector & cascade

✗ Dùng `#id` để tạo kiểu → điểm 1-0-0, sau này không ghi đè nổi	✓ Dùng class; ID chỉ để làm neo link
✗ Thêm `!important` mỗi khi CSS "không ăn"	✓ Mở DevTools → Styles: tìm rule bị gạch ngang và lý do
✗ Viết `@import` giữa tệp → bị bỏ qua, không báo lỗi	✓ `@layer` rồi `@import` ở **đầu** tệp
✗ Đặt `margin` ở `body` mong các thẻ con kế thừa	✓ `margin` không kế thừa; đặt cho đúng phần tử

2.1 Thử ngay: ai thắng?

THỬ NGAY · 5 PHÚT

1. Tính specificity: `.card:has(.badge--hot)` · `.field input:focus-visible` · `:where(h1, p)`
2. Cùng chọn link "Sự kiện" trên menu: `a { color: blue }` (lớp `base`) và `.site-nav__link { color: white }` (lớp `components`). Chữ màu gì?
3. Đổi thứ tự dòng `@layer` thành `components, base`. Kết quả câu 2 thay đổi không?
4. Mở `events.html` (solution) → DevTools → chọn link menu → tab *Styles*: rule nào bị gạch?

Đáp án gợi ý — Thử ngay 2.1

- `.card:has(.badge--hot)` = **0-2-0** (`:has()` lấy điểm của thứ bên trong) · `.field input:focus-visible` = **0-2-1** · `:where(…)` = **0-0-0**.
- **Trắng**: lớp `components` khai báo sau nên thắng (và điểm cũng cao hơn).
- Có: khi đó `base` là lớp sau → `a { color: blue }` thắng dù điểm thấp hơn. Lớp quyết định *trước* specificity.
- Rule `a { color: … }` của lớp base bị gạch; DevTools ghi tên lớp cạnh mỗi rule.

HỎI THÊM Vì sao solution dùng `:where()` để reset margin thay vì viết thẳng `body, h1, p { … }`?

2.2

PHẦN 2 / 7

PHẦN 2

Box model, đơn vị, màu & font

- Box model và `box-sizing`
- Đơn vị px, rem, %, vw, `clamp()`
- Màu: hex, `rgb()`, `oklch()`
- Font stack và `font-display`

2.2 Ví dụ đời thường: khung tranh treo tường

🖼️ Bức tranh
Phần vẽ chính = **content** (chữ, ảnh).

⬜ Viền giấy trắng
Khoảng đệm quanh tranh, cùng màu nền tranh = **padding**.

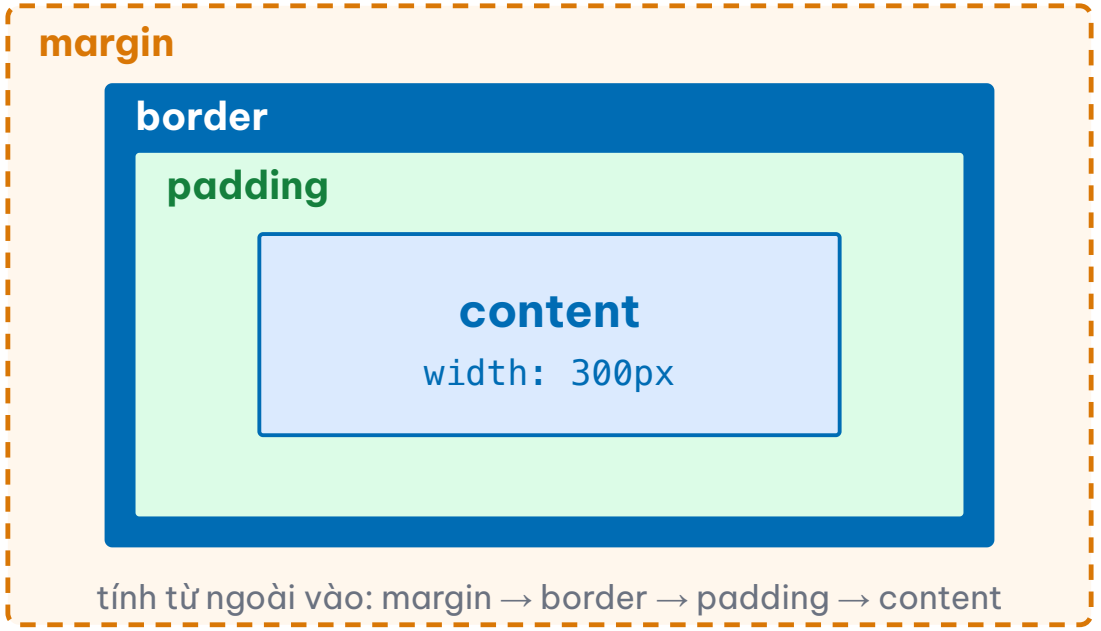
🟫 Khung gỗ
Đường bao ngoài cùng của tranh = **border**.

↔ Khoảng cách tới tranh bên cạnh
Chỗ trống bên ngoài khung = **margin**.

ACTIVE RECALL

Treo tranh vào chỗ tường rộng đúng 60 cm.
Bạn đo bề rộng của tranh, hay của cả khung?

2.2 Box model — mọi phần tử là một cái hộp



content
chữ, ảnh thật sự

padding
đệm trong viền, có nền

border
đường viền

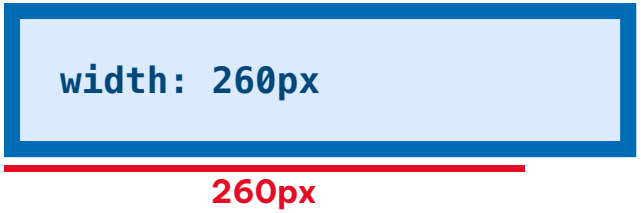
margin
khoảng trống bên ngoài

Hình 2.6. Bốn lớp của box model (tự vẽ).

Xem thử: DevTools → chọn phần tử → tab *Computed* có đúng hình hộp này kèm số px.

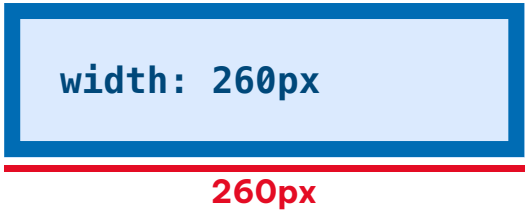
2.2 `box-sizing`: width tính tới đâu?

content-box (mặc định)



Hộp thật rộng **260 + 2×20 + 2×8 = 316px**

border-box



Hộp thật rộng **đúng 260px**, content tự co

```css
/* Tính width/height gồm cả padding và border → dễ đoán kích thước hơn */
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Đoạn mã 2.4. Áp `border-box` cho mọi phần tử. Nguồn: `02_Projects/ch02/solution/css/base.css`

Hình 2.7. Hai hộp trên là HTML + CSS chạy thật: `padding: 20px; border: 8px`.

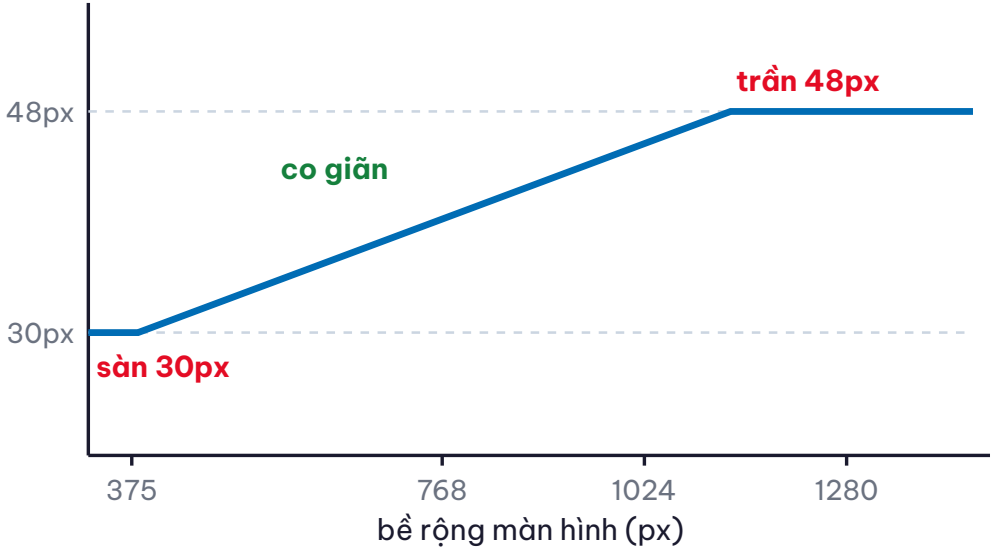
2.2 Đơn vị đo: px, rem, %, vw, ch

Bảng 2.2. Các đơn vị đo thường dùng và chỗ dùng trong EduHub

ĐƠN VỊ	NGHĨA	DÙNG TRONG EDUHUB
`px`	Điểm ảnh cố định	Viền mỏng `1px`, bóng đổ
`rem`	Bội số cỡ chữ gốc (mặc định 16px)	Khoảng cách `--space-4: 1rem`, breakpoint `40rem`
`%`	Phần trăm của phần tử cha	`max-inline-size: 100%` cho ảnh
`vw` / `vh`	1% bề rộng / chiều cao màn hình	Trong `clamp()`, `min-block-size: 100vh`
`ch`	Bề rộng chữ "0"	`max-inline-size: 60ch` → dòng chữ dễ đọc

ACTIVE RECALL Người dùng tăng cỡ chữ trình duyệt lên 20px. Khoảng cách `1rem` thành bao nhiêu px? `16px` thì sao?

2.2 `clamp()` — chữ to dần theo màn hình




```
clamp(
nhỏ nhất,
ưu tiên (co giãn),
lớn nhất
)
```

h1 của EduHub

Hình 2.8. Đồ thị cỡ chữ h1 theo bề rộng màn hình với clamp (tự vẽ).

```
--text-3xl: clamp(1.875rem, 1.3rem + 2.4vw, 3rem);
```

Đoạn mã 2.5. Token cỡ chữ của `h1` dùng `clamp()`. Nguồn: `02_Projects/ch02/solution/css/tokens.css`

2.2 Màu trong CSS: hex, rgb(), color-mix(), oklch()

Bảng 2.3. Bốn cách viết màu trong CSS

CÁCH VIẾT	VÍ DỤ	GHI CHÚ
Hex	`#003f87`	Palette EduHub trong `tokens.css`
`rgb()` + độ trong	`rgb(0 29 69 / 0.08)`	Bóng đổ `--shadow-sm`
`color-mix()`	`color-mix(in srgb, var(--color-surface-muted) 40%, transparent)`	Hàng chẵn của bảng
`oklch()`	`oklch(38% 0.134 257)` ≈ `#003f87`	Sáng · đậm · sắc độ — chỉnh độ sáng "đều mắt" hơn

ACTIVE RECALL Muốn màu xanh chính **sáng hơn**: sửa số nào trong `oklch()`?

Hex, rgb(), color-mix() có trong solution (`tokens.css`, `components.css`).
`oklch` là minh hoạ — solution giữ hex để khớp palette Ch1.

```
--font-sans: "Be Vietnam Pro", system-ui, -apple-system, "Segoe UI", Roboto,
  "Helvetica Neue", Arial, sans-serif;
--font-mono: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
```

Đoạn mã 2.6. Font stack trong `tokens.css`. Nguồn: `02_Projects/ch02/solution/css/tokens.css`

EduHub **không tải font ngoài**: máy có Be Vietnam Pro thì dùng, không thì font hệ thống → nhanh, chạy cả khi mất mạng. Danh sách nhiều font = **font stack**: thiếu font trước thì lấy font sau.

```
@font-face {
  font-family: "Be Vietnam Pro";
  src: url("fonts/bvp.woff2") format("woff2");
  font-display: swap;
}
```

Đoạn mã 2.7. Minh hoạ tự tải font — solution Ch2 không dùng.

`font-display: swap`: hiện chữ bằng font dự phòng ngay, tải xong mới đổi → không bị "trắng chữ". Google Fonts bật sẵn bằng tham số `&display=swap` trong URL.

2.2 Lỗi hay gặp — box model & đơn vị

✗ `width: 100%` + `padding` → tràn ngang, hiện thanh cuộn

→ ✓ `box-sizing: border-box` cho mọi phần tử

✗ Cỡ chữ, khoảng cách viết toàn `px`

→ ✓ `rem` để tôn trọng cỡ chữ người dùng chọn

✗ `font-size: 5vw` → điện thoại chữ tí hon, màn to chữ khổng lồ

→ ✓ `clamp(min, …vw, max)` có sàn và trần

✗ Tải 6 kiểu font từ Google Fonts "cho chắc"

→ ✓ Chỉ tải đúng độ đậm dùng tới, kèm `display=swap`

2.2 Thử ngay: đo cái hộp

THỬ NGAY · 5 PHÚT

1. Một nút có `width: 200px; padding: 0 24px; border: 2px solid`. Bề rộng thật với `content-box`? với `border-box`?
2. Mở `index.html` (solution) → DevTools → chọn nút "Xem lịch sự kiện" → tab *Computed*: đọc padding, border, chiều cao.
3. Dùng thanh kéo của Device Toolbar từ 375 → 1280px: tiêu đề `h1` to lên dần rồi dừng ở bao nhiêu px?
4. Trong Styles, tạm bỏ tick `box-sizing: border-box` ở rule `*`: cái gì vỡ đầu tiên?

2.2 Đáp án gợi ý — Thử ngay 2.2

- `content-box`: 200 + 2×24 + 2×2 = **252px**. `border-box`: đúng **200px**.
- Nút `.btn`: `min-block-size: 2.75rem` → cao ít nhất **44px** — vừa ngón tay (ch2-T3).
- `h1` dùng `--text-3xl`: từ 30px, chạm trần **48px** khi màn hình khoảng 1133px.
- Ô nhập ở `register.html` (`inline-size: 100%` + padding) rộng thêm phần padding + viền → lòi ra khỏi khung form.

HỎI THÊM Vì sao solution dùng `inline-size` thay cho `width`? (gợi ý: chiều viết chữ)

2.3

PHẦN 3 / 7

PHẦN 3

Flexbox — dàn hàng một chiều

- Trục chính và trục phụ
- `justify-content`, `align-items`, `gap`
- Header và menu của EduHub
- Card cao bằng nhau

2.3 Ví dụ đời thường: xếp hàng chào cờ

➡️ Một hàng ngang

Các bạn đứng thành **một hàng** — Flexbox dàn phần tử theo một chiều.

↔ Giãn đều

Lớp trưởng hô "giãn cách đều!" — giống `justify-content: space-between`.

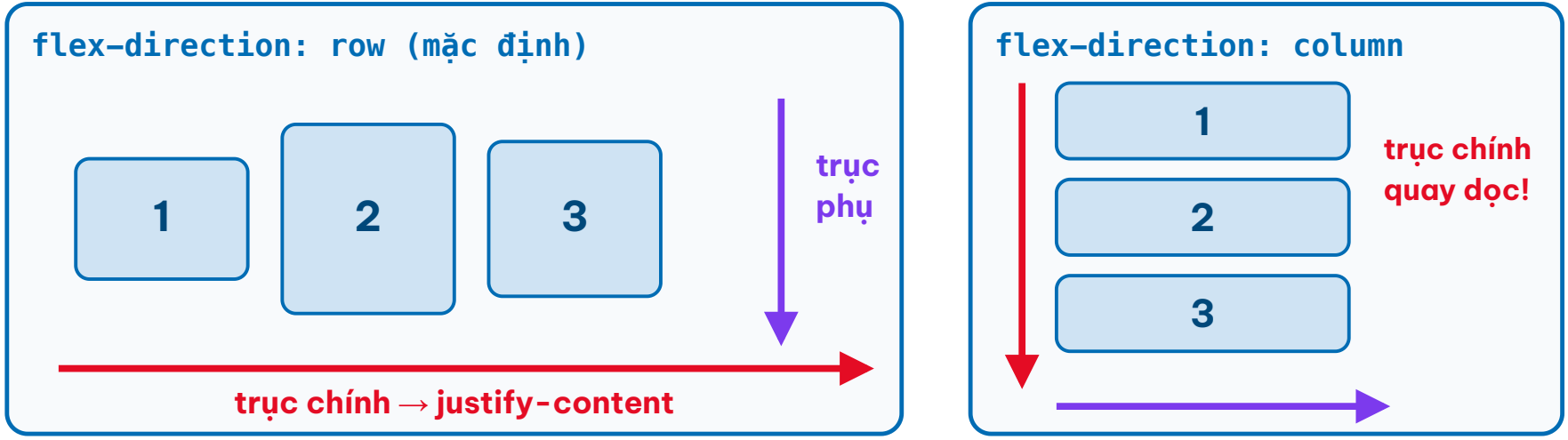
↕ Thẳng hàng

Cao thấp khác nhau nhưng thẳng hàng ở giữa — `align-items: center`.

↩️ Hết chỗ thì xuống hàng

Sân hẹp, hàng dài quá thì đứng thêm hàng sau — `flex-wrap: wrap`.

ACTIVE RECALL Menu của EduHub (Trang chủ · Sự kiện · CLB · Đăng ký) giống "hàng chào cờ" ở điểm nào?



justify-content căn theo trục chính · align-items căn theo trục phụ · đổi direction là hai trục đổi chỗ

Hình 2.9. Trục chính và trục phụ của Flexbox (tự vẽ).

Bật bằng `display: flex` ở **cha** (flex container); các con trực tiếp thành **flex item**.

2.3 Flexbox: thuộc tính hay dùng

Bảng 2.4. Thuộc tính đặt ở cha

ĐẶT Ở CHA (CONTAINER)	
`display: flex`	Bật Flexbox
`flex-direction`	`row` | `column`
`flex-wrap: wrap`	Hết chỗ thì xuống dòng
`justify-content`	Căn trục chính
`align-items`	Căn trục phụ
`gap`	Khoảng cách giữa các con

Bảng 2.5. Thuộc tính đặt ở con

ĐẶT Ở CON (ITEM)	
`flex: 1`	Giãn ra lấp chỗ trống
`margin-block-start: auto`	Đẩy item xuống cuối (trong cột)
`align-self`	Căn riêng một item
`order`	Đổi thứ tự hiển thị (cẩn thận a11y)

ACTIVE RECALL Muốn logo nằm trái, menu nằm phải trên cùng một hàng: đặt thuộc tính gì, ở cha hay con?

2.3 Minh hoạ trực quan: `justify-content` & `align-items`

`justify-content: flex-start`

`justify-content: center`

`justify-content: space-between`

`align-items: center`

`flex-wrap: wrap` + `gap`



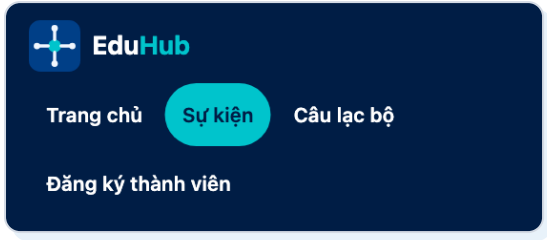
Hình 2.10. Mỗi hàng là một `div` có `display: flex` thật — mở DevTools sửa giá trị để xem đổi ngay.

Header EduHub bằng Flexbox

```css
.site-header__inner {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: var(--space-2) var(--space-4);
  padding-block: var(--space-3);
}
```

```css
.site-nav__list {
  display: flex;
  flex-wrap: wrap;
  gap: var(--space-1);
  padding: 0;
  list-style: none;
}
```

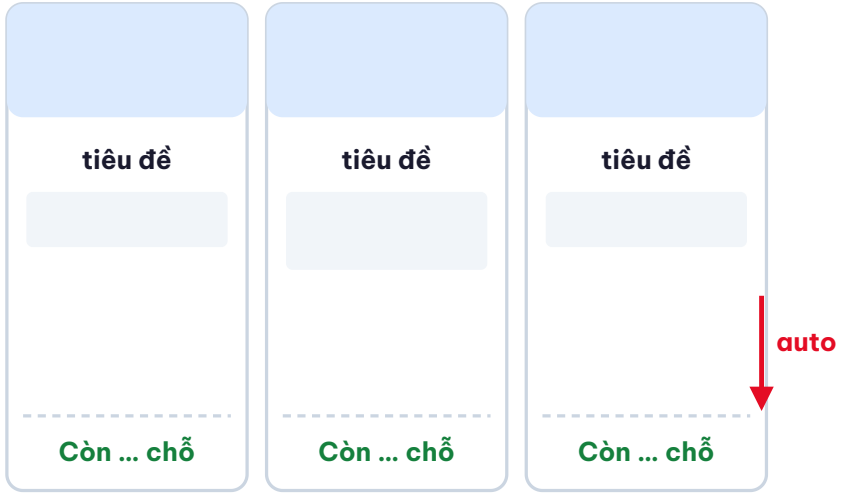
Đoạn mã 2.8. Header và menu dàn bằng Flexbox. Nguồn: `02_Projects/ch02/solution/css/layout.css` · đã lược chú thích



Hình 2.11. Menu xuống dòng ở 375px (ảnh chụp solution).

- `space-between`: logo dạt trái, menu dạt phải
- `flex-wrap: wrap`: hết chỗ thì menu rớt xuống dòng — không cần media query (ch2-T3)
- `gap` hai giá trị: khoảng cách hàng · cột

2.3 Card cao bằng nhau, “Còn … chỗ” nằm đáy



Hình 2.12. Ba card cùng hàng, dòng số chỗ bị đẩy xuống đáy (tự vẽ).

```css
.card__body {
  display: flex;
  flex: 1;
  flex-direction: column;
  …
}

.event-card__seats {
  margin-block-start: auto;
  padding-block-start: var(--space-3);
  …
}
```

Đoạn mã 2.9. Thân card là cột flex, dòng số chỗ nằm đáy. Nguồn: `02_Projects/ch02/solution/css/components.css` (ch2-T4) · đã lược chú thích

`.card__body` là cột flex giãn hết card; `margin-block-start: auto` “ăn” hết chỗ trống phía trên dòng số chỗ.

2.3 Lỗi hay gặp – Flexbox

✗ Đặt `justify-content` ở **con**	✓ Đặt ở **cha** có `display: flex`
✗ Đổi `flex-direction: column` rồi vẫn dùng `justify-content` để căn ngang	✓ Cột thì trục chính dọc → căn ngang bằng `align-items`
✗ Menu nhiều mục tràn ngang trên điện thoại	✓ `flex-wrap: wrap` + `gap`
✗ Dùng `margin-right` cho từng mục để tạo khoảng cách	✓ `gap` ở cha – không thừa khoảng ở mục cuối

2.3 Thử ngay: điều khiển header

THỬ NGAY · 5 PHÚT

1. Mở `index.html` (solution) → DevTools → chọn `.site-header__inner` (có nhãn `flex`).
2. Bấm biểu tượng *Flexbox editor* cạnh `display: flex`, thử `center`, `space-around`.
3. Bỏ tick `flex-wrap: wrap` rồi thu màn hình về 375px: chuyện gì xảy ra?
4. Trên `events.html`, bỏ tick `margin-block-start: auto` của `.event-card__seats`: dòng số chỗ nằm đâu?

- Gợi ý: bỏ `wrap` → menu chen chúc, trang có thanh cuộn ngang. Bỏ `auto` → dòng số chỗ dính ngay dưới mô tả, card cao thấp lệch nhau.

2.4

PHẦN 4 / 7

PHẦN 4

Grid — lưới hai chiều

- Track, line, cell và đơn vị `fr`
- `grid-template-areas` cho trang chi tiết
- `auto-fit` + `minmax()` · subgrid
- Flexbox hay Grid?

2.4 Ví dụ đời thường: sơ đồ chỗ ngồi phòng thi

🪑 Hàng và cột

Phòng thi kẻ sẵn **hàng × cột**; mỗi bạn ngồi đúng ô — Grid điều khiển **hai chiều** cùng lúc.

👩‍🏫 Bàn giám thị

Bàn giám thị chiếm **cả hàng đầu** — một phần tử trải nhiều ô (vùng *area*).

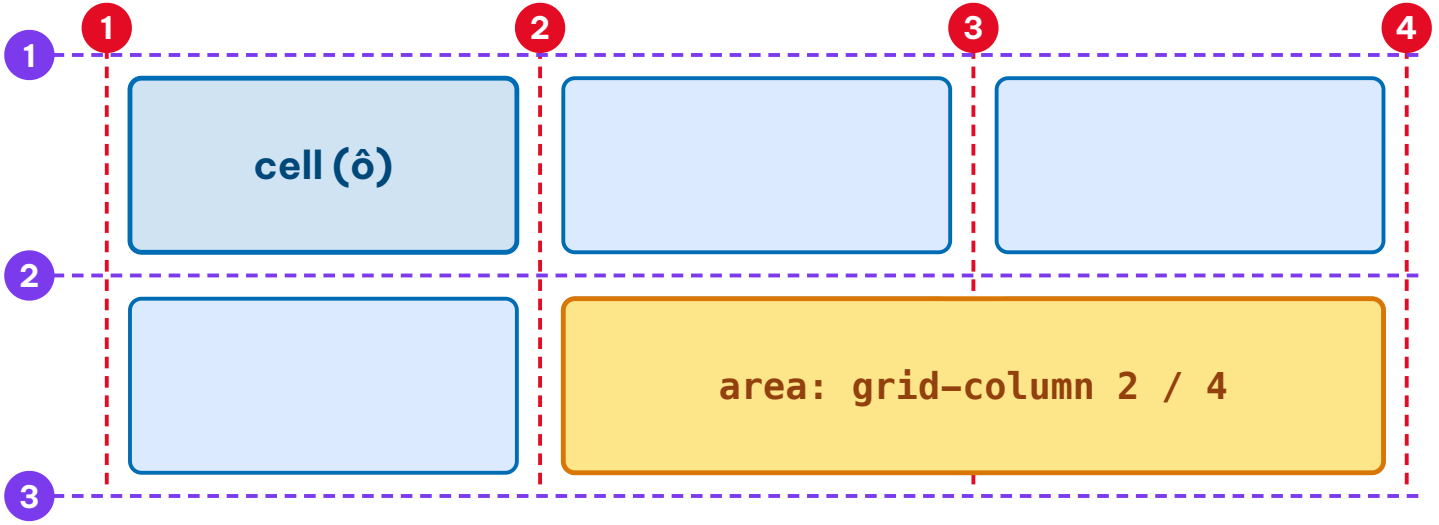
🚶 Lối đi

Khoảng trống cố định giữa các dãy bàn = `gap`.

📐 Phòng to, phòng nhỏ

Phòng rộng kê nhiều cột, phòng hẹp ít cột — `auto-fit`.

ACTIVE RECALL Flexbox ở 2.3 là "một hàng". Bố cục nào trên EduHub cần cả hàng lẫn cột?



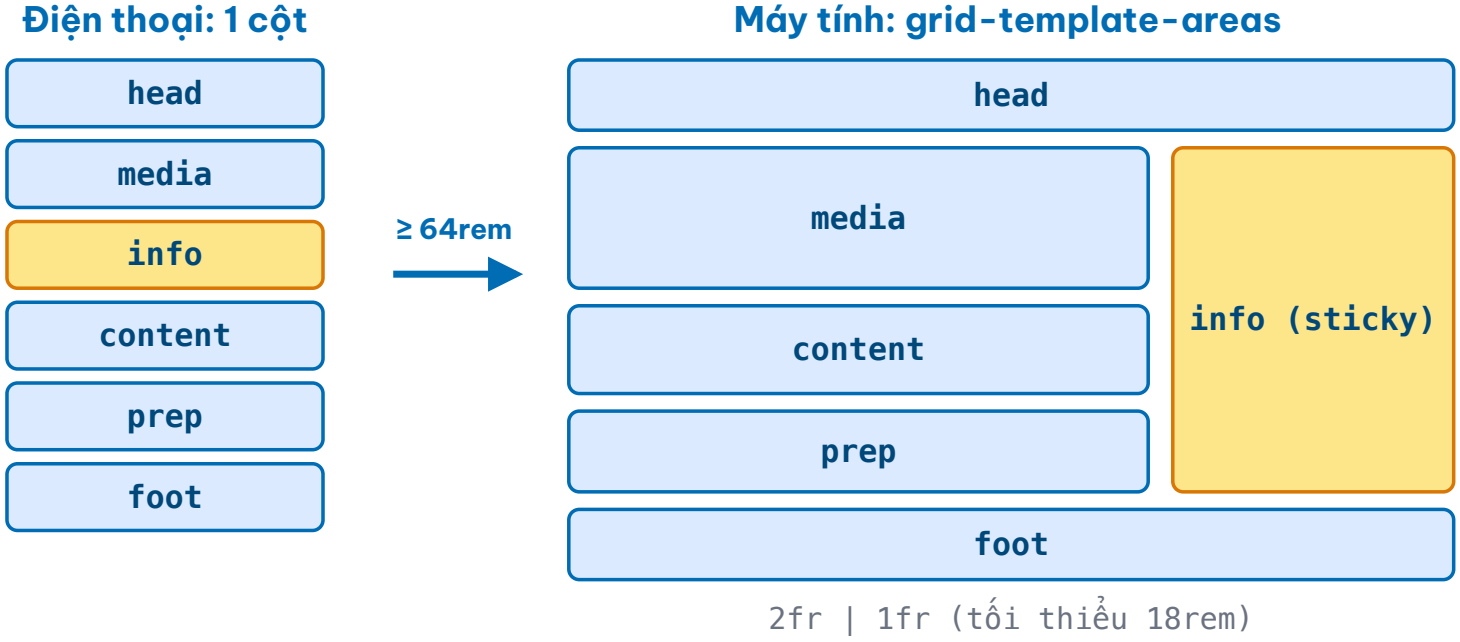
Đường kẻ (line) đánh số từ 1 · cột/hàng = track · khoảng giữa = gap

`grid-template-columns: 1fr 1fr 1fr` → ba cột chia đều phần còn lại

Hình 2.13. Lưới 3 cột 2 hàng với số đường kẻ, ô và vùng (tự vẽ).

`fr` = "một phần" chỗ trống còn lại. `2fr 1fr` → cột trái gấp đôi cột phải.

Grid template areas — vẽ bố cục bằng chữ



Hình 2.14. Bố cục trang event-detail.html: 1 cột trên điện thoại, 2 cột có vùng info bên phải trên máy tính (tự vẽ).

2.4 Code: khai báo vùng cho `.detail`

```css
.detail {
  display: grid;
  gap: var(--space-6);

  @media (width >= 64rem) {
    grid-template-columns: minmax(0, 2fr) minmax(18rem, 1fr);
    grid-template-areas:
      "head    head"
      "media   info"
      "content info"
      "prep    info"
      "foot    foot";
    column-gap: var(--space-8);
  }
}
```

Đoạn mã 2.10. Khai báo vùng lưới cho trang chi tiết. Nguồn: `02_Projects/ch02/solution/css/layout.css`

Mỗi chuỗi = **một hàng**; tên lặp lại = vùng trải nhiều ô.

Code: gán phần tử vào vùng

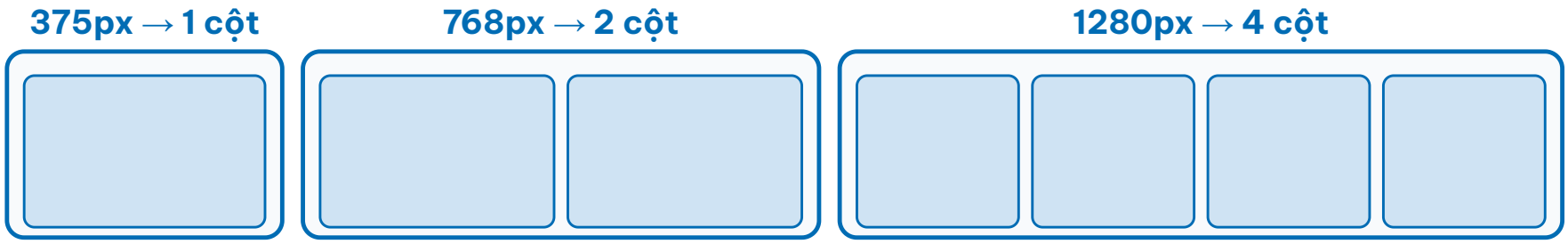
```css
/* Gán vùng chỉ trong @media: tên vùng không tồn tại ở 1 cột sẽ sinh ra cột ngầm thừa */
@media (width >= 64rem) {
  .detail__head    { grid-area: head; }
  .detail__media   { grid-area: media; }
  .detail__content { grid-area: content; }
  .detail__prep    { grid-area: prep; }
  .detail__foot    { grid-area: foot; }

  .detail__info {
    grid-area: info;
    align-self: start;
    position: sticky;            /* cột phải đứng yên khi cuộn nội dung */
    inset-block-start: 6rem;     /* chừa chỗ cho header sticky */
  }
}
```

Đoạn mã 2.11. Gán từng phần tử vào vùng. Nguồn: `02_Projects/ch02/solution/css/layout.css`

`sticky`: hộp "Thông tin chung" đứng yên khi cuộn (ch2-T5).

2.4 `auto-fit` + `minmax()`: lưới tự đổi số cột



`repeat(auto-fit, minmax(min(16rem, 100%), 1fr))`

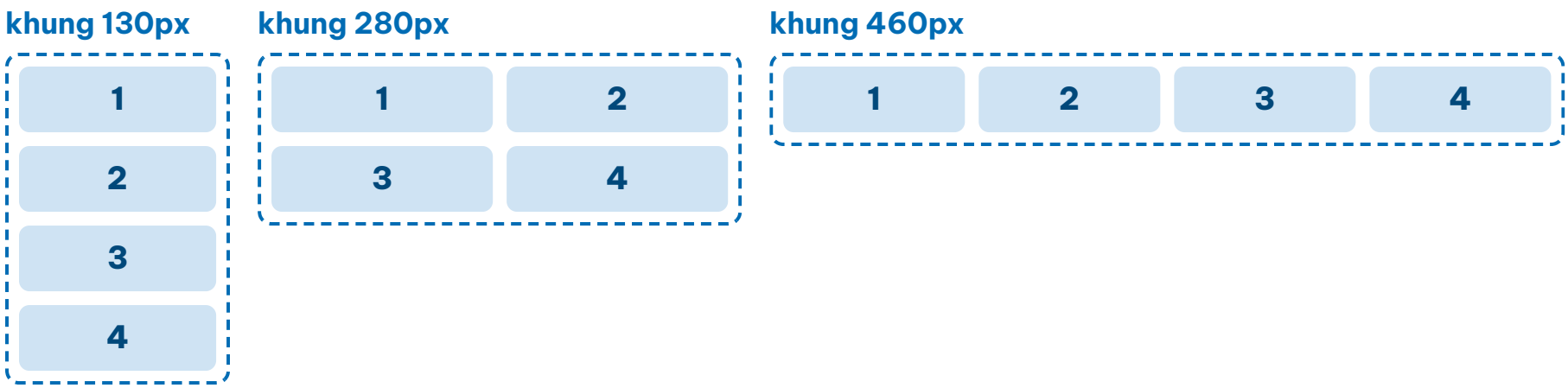
mỗi cột ≥ 16rem · còn chỗ thì thêm cột · thiếu chỗ thì xuống hàng · không cần media query

Hình 2.15. Lưới card 1, 2 và 4 cột ở 375, 768 và 1280 px (tự vẽ).

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr));
  gap: var(--space-6);
  padding: 0;
  list-style: none;
}
```

Đoạn mã 2.12. Lưới card tự đổi số cột. Nguồn: `02_Projects/ch02/solution/css/layout.css`

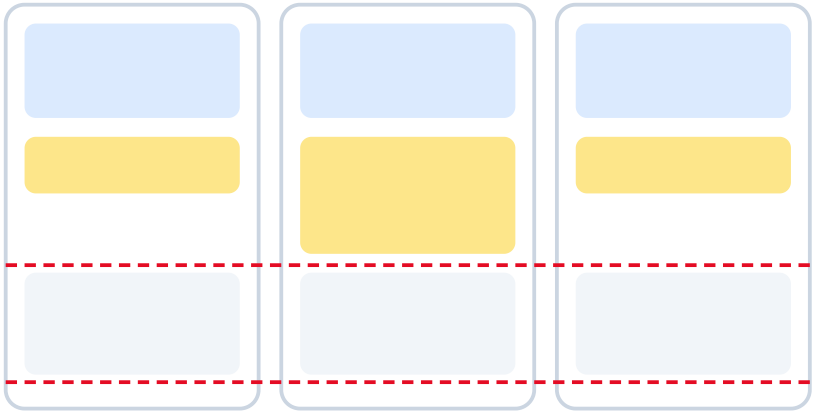
2.4 Minh hoạ trực quan: cùng một CSS, ba bề rộng




```
.gd-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100px, 100%), 1fr));
  gap: 8px;
}
```

Hình 2.16. Lưới thu nhỏ (100px thay cho 16rem) chạy thật trên slide — cùng công thức với `.card-grid`.

2.4 subgrid — hàng con thẳng theo lưới cha



Hình 2.17. Ba card dùng subgrid nên các phần thẳng hàng (tự vẽ).

```
.card-grid { display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(16rem, 1fr)); }

.card {
  display: grid;
  grid-row: span 4;
  grid-template-rows: subgrid;
}
```

Đoạn mã 2.13. Minh hoạ subgrid — solution dùng Flexbox trong card (2.3) cho đơn giản.

`subgrid`: lưới con **mượn** đường kẻ của lưới cha → tiêu đề, mô tả của các card khác nhau vẫn thẳng hàng.

Flexbox hay Grid?

Bảng 2.6. So sánh Flexbox và Grid

	FLEXBOX	GRID
Số chiều	Một chiều: hàng *hoặc* cột	Hai chiều: hàng *và* cột
Ai quyết định kích thước	Nội dung đẩy ra ngoài	Lưới định sẵn, nội dung xếp vào
Trong EduHub	Header, menu, nhóm nút, thân card, radio "viên thuốc"	Lưới card, trang chi tiết, form 2 cột, `<dl>` 2 cột
Mẹo nhớ	"Xếp hàng"	"Kẻ ô bàn cờ"

ACTIVE RECALL Form đăng ký: các ô xếp 2 cột từ 640px. Solution dùng Flexbox hay Grid? Vì sao?

Hai cách không loại trừ nhau: card nằm trong **Grid**, bên trong card lại là **Flexbox**.

2.4 Lỗi hay gặp – Grid

✗ Gán `grid-area` ở ngoài `@media` → ở 1 cột sinh cột ngầm thừa

→ ✓ Chỉ gán vùng trong `@media` nơi có `grid-template-areas`

✗ `minmax(16rem, 1fr)` → màn 240px vẫn tràn ngang

→ ✓ `minmax(min(16rem, 100%), 1fr)`

✗ Chuỗi areas thiếu ô hoặc vùng không thành hình chữ nhật

→ ✓ Mỗi hàng đủ số cột; vùng phải là hình chữ nhật liền

✗ Thêm 3 media query để đổi 1→2→3 cột cho lưới card

→ ✓ `auto-fit` + `minmax` tự xử lý

2.4 Thử ngay: vẽ lại trang chi tiết

THỬ NGAY · 5 PHÚT

1. Mở `event-detail.html` (solution) ở 1280px → DevTools → nhãn `grid` cạnh `.detail` → bật *Show area names*.
2. Sửa `grid-template-areas` để cột `info` nằm **bên trái**. Cần đổi thêm gì ở `grid-template-columns`?
3. Trên `events.html`, đổi `16rem` thành `22rem`: ở 1280px còn mấy cột?

- Gợi ý: đảo tên trong chuỗi (`"info media"`…) và đảo cột thành `minmax(18rem, 1fr) minmax(0, 2fr)`. Với 22rem (352px): (1104 + 24) / (352 + 24) ≈ 3 → **3 cột**.

2.5

PHẦN 5 / 7

PHẦN 5

Responsive mobile-first

- Mobile-first và hai breakpoint
- Media query kiểu range, lồng trong rule
- Ảnh và bảng responsive
- Mở rộng: container queries

2.5 Ví dụ đời thường: soạn vali đi chơi

🎒 Ba lô trước

Xếp đồ **thiết yếu** vào ba lô nhỏ — giao diện điện thoại, viết đầu tiên.

💧 Nước trong bình

Nước lấy hình cái bình — bố cục co giãn theo bề rộng màn hình.

🧳 Có vali to hơn

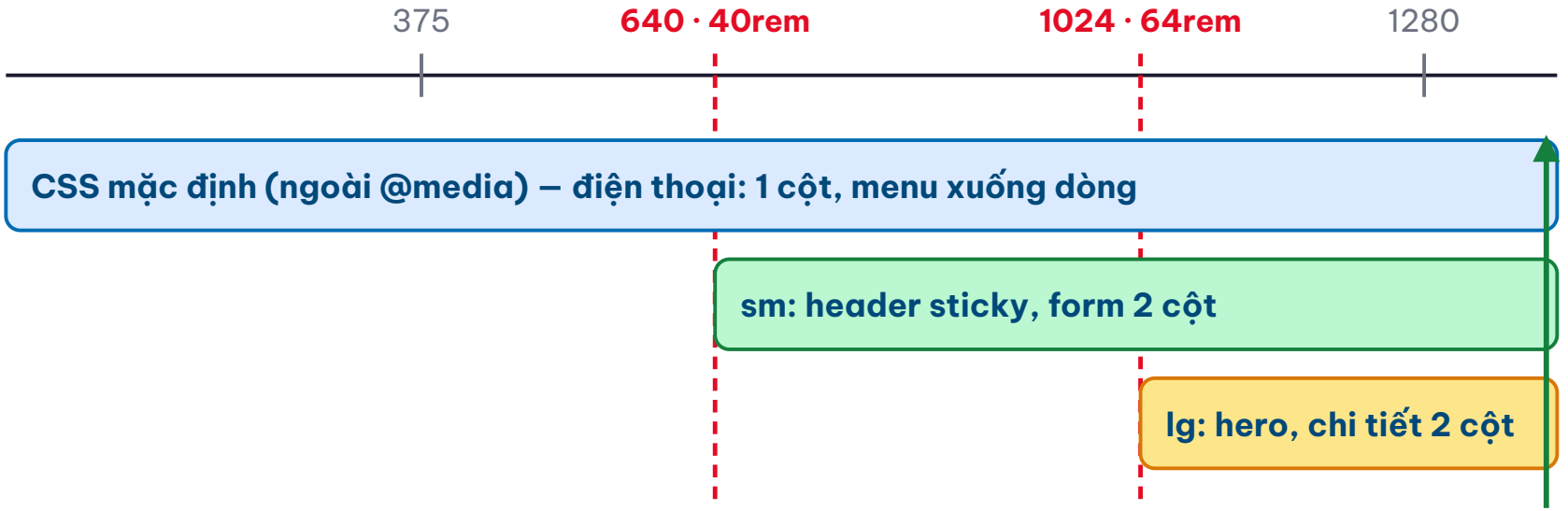
Có thêm chỗ thì **bổ sung** đồ — media query `width >= …` thêm cột, thêm khoảng trắng.

📱 Hơn nửa người dùng

Sinh viên xem lịch sự kiện chủ yếu bằng **điện thoại**.

ACTIVE RECALL Làm giao diện máy tính trước rồi "nhét" vào điện thoại — dễ hay khó hơn? Vì sao?

2.5 Mobile-first: 2 breakpoint của EduHub



Mỗi tầng chỉ THÊM/ĐỔI vài dòng; màn rộng nhận cả 3 tầng cộng dồn

Hình 2.18. Thước bề rộng màn hình với hai mốc 640 và 1024 px, các tầng CSS cộng dồn (tự vẽ).

Bắt buộc có trong HTML (đã có từ Ch1): `<meta name="viewport" content="width=device-width, initial-scale=1">`

Media query: cú pháp range + lồng trong rule

```css
.container {
  inline-size: 100%;
  max-inline-size: var(--container-max);
  margin-inline: auto;
  padding-inline: var(--space-4);

  @media (width >= 40rem) {
    padding-inline: var(--space-6);
  }
}
```

```css
.site-header {
  z-index: 10;
  background-color: var(--color-header-bg);
  color: var(--color-header-text);
  box-shadow: var(--shadow-md);

  @media (width >= 40rem) {
    position: sticky;
    inset-block-start: 0;
  }
}
```

Đoạn mã 2.14. Media query lồng trong rule, viết kiểu mobile-first.
Nguồn: `02_Projects/ch02/solution/css/layout.css` · đã lược chú thích

- `width >= 40rem` (cú pháp range) dễ đọc hơn `min-width: 40rem` — cùng nghĩa.
- Viết `@media` **ngay trong rule** nhờ nesting gốc (2.6) → mọi thứ của `.container` nằm một chỗ.

```css
img,
svg {
  display: block;
  max-inline-size: 100%;
  block-size: auto;
}

.card__media {
  inline-size: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
}
```

Đoạn mã 2.15. Ảnh không tràn khung, ảnh card tỉ lệ 16:9. Nguồn: `css/base.css`, `css/components.css`

```html
<img src="hero-800.jpg"
  srcset="hero-400.jpg 400w,
          hero-800.jpg 800w,
          hero-1600.jpg 1600w"
  sizes="(min-width: 1024px) 50vw, 100vw"
  width="800" height="450" alt="…">
```

Đoạn mã 2.16. Minh hoạ `srcset` / `sizes` — EduHub dùng ảnh SVG nên không cần.

- `srcset`: trình duyệt tự chọn ảnh vừa đủ nét
- `width/height`: giữ chỗ, trang không "nhảy"

2.5 Bảng dài trên điện thoại: cuộn trong khung riêng

```css
.table-wrap {
  overflow-x: auto;
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  background-color: var(--color-surface);
}

.table {
  inline-size: 100%;
  min-inline-size: 40rem;
  border-collapse: collapse;
  …
}
```

Đoạn mã 2.17. Bảng cuộn ngang trong khung riêng.
Nguồn: `02_Projects/ch02/solution/css/components.css`

Bảng quá rộng cho 375px: **chỉ bảng** cuộn ngang, trang thì không (ch2-T5).

Sự kiện sắp tới của các câu lạc bộ (giờ Việt Nam)

Thời gian	Sự kiện
07:30, T7 17/10/2026	Workshop: Làm website đầu tiên với HTML
07:00, T7 24/10/2026	Ngày hội Hiến máu tình nguyện
18:30, T7 31/10/2026	English Speaking Night: Halloween
07:00, CN 08/11/2026	Giải bóng đá sinh viên Khoa Công nghệ và Kỹ

Hình 2.19. Bảng ở 375px cuộn trong khung (ảnh chụp solution).

Container queries — card nhìn khung chứa nó



Hình 2.20. Cùng card, khung rộng thì nằm ngang, khung hẹp thì nằm dọc (tự vẽ).

```
.card-slot { container-type: inline-size; }

@container (width >= 28rem) {
  .card { flex-direction: row; }
}
```

Đoạn mã 2.18. Minh hoạ, không tính điểm.

`@media` hỏi "**màn hình** rộng bao nhiêu?".
`@container` hỏi "**khung chứa** rộng bao nhiêu?"
→ một component dùng được ở mọi chỗ.

2.5 Lỗi hay gặp — responsive

✗ Viết desktop trước, rồi `max-width` để "sửa" cho điện thoại	✓ Mobile-first: mặc định = điện thoại, `width >= …` để mở rộng
✗ Mỗi trang một bộ breakpoint 600, 768, 992, 1200…	✓ Đúng 2 mốc chung `40rem`, `64rem` (ghi trong `tokens.css`)
✗ Quên `<meta name="viewport">` → điện thoại hiện bản thu nhỏ	✓ Luôn có meta viewport trong `<head>`
✗ `fieldset`/bảng làm cả trang cuộn ngang	✓ `min-inline-size: 0` cho fieldset; bảng bọc `.table-wrap`

THỬ NGAY · 5 PHÚT

1. Mở 4 trang solution, bật Device Toolbar, thử lần lượt **375 · 768 · 1280px**.
2. Ở mỗi bề rộng, chạy trong Console: `document.documentElement.scrollWidth <= innerWidth`.
3. Ở `register.html`, xoá `min-inline-size: 0` của `fieldset`, thu về 320px: kết quả câu 2 đổi thế nào?
4. Kéo bề rộng qua mốc 640px và 1024px: ghi lại 2 thay đổi bạn thấy ở mỗi mốc.

2.5 Đáp án gợi ý — Thử ngay 2.5

- Cả 4 trang trả về `true` ở cả 3 bề rộng (tiêu chí chung của bài Ch2).
- Thiếu `min-inline-size: 0`: fieldset giữ bề rộng tối thiểu theo nội dung → có thể `false`, xuất hiện cuộn ngang.
- Qua **640px**: header dính đầu trang khi cuộn, form thành 2 cột, link menu chữ to hơn.
- Qua **1024px**: hero 2 cột (chữ + ảnh), trang chi tiết có cột "Thông tin chung" sticky, footer thành một hàng.

HỎI THÊM Lưới card đổi số cột ở mốc nào? (gợi ý: câu hỏi bẫy)

2.6

PHẦN 6 / 7

PHẦN 6

CSS hiện đại

- Design token bằng custom properties
- Dark mode với `prefers-color-scheme`
- Nesting, `:has()`, `:is()` / `:where()`
- Transition, animation, reduced-motion

2.6 Ví dụ đời thường: bảng mã màu sơn

🎨 Đặt tên màu

Thợ sơn ghi "Xanh EduHub = mã 003F87" một lần — giống **custom property** `--palette-blue-700`.

🏷️ Gọi theo vai trò

"Màu cửa chính", "màu tường" — giống token `--color-primary`, `--color-bg`.

🌙 Đổi một chỗ, cả nhà đổi

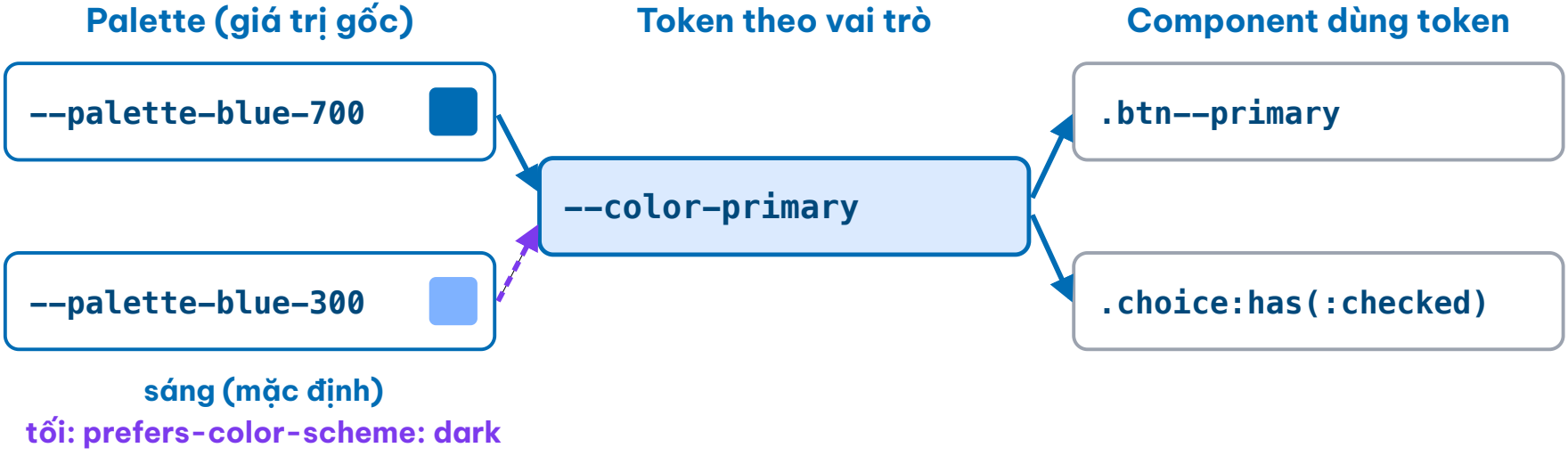
Đổi "màu tường" sang tối → mọi bức tường tối theo. Đó là **dark mode**.

📏 Thước dùng chung

Khoảng cách, bo góc cũng đặt tên: `--space-4`, `--radius-md`.

ACTIVE RECALL Nếu mã màu `#003f87` viết thẳng ở 40 chỗ trong CSS, muốn đổi màu thương hiệu thì mất bao lâu?

2.6 Design token: 3 tầng biến



Hình 2.21. Palette trỏ vào token ngữ nghĩa, component chỉ đọc token (tự vẽ).

2.6 Custom properties trong `tokens.css`

```css
:root {
  --palette-blue-900: #001d45;
  --palette-blue-700: #003f87;
  …
  --color-primary: var(--palette-blue-700);
  …
  --space-4: 1rem;
  …
}
```

```css
.btn--primary {
  background-color: var(--color-primary);
  color: var(--color-on-primary);
  …
}
```

Đoạn mã 2.19. Khai báo token và dùng token trong component.
Nguồn: `css/tokens.css` (đã lược chú thích), `css/components.css`

- Khai báo: `--ten-bien: giá trị;` trong `:root` (gốc trang) → dùng được ở mọi nơi.
- Đọc: `var(--ten-bien)`. Component **không** viết mã màu trực tiếp (ch2-T1).

2.6 Dark mode bằng `prefers-color-scheme`

```
@media (prefers-color-scheme: dark) {
  :root {
    --color-primary: var(--palette-blue-300);
    --color-primary-hover: #a9cbff;
    --color-on-primary: var(--palette-blue-900);
    --color-accent-strong: var(--palette-cyan-500);
    --color-bg: #000f26;
    …
  }
}
```

Đoạn mã 2.20. Dark mode chỉ đổi giá trị token.
Nguồn: `02_Projects/ch02/solution/css/tokens.css`



Hình 2.22. Cùng một trang khi máy để sáng và tối (ảnh chụp solution).

- Máy ở chế độ tối → khối `@media` bật → token đổi giá trị → cả trang tối theo
- `color-scheme: light dark` trong `:root`: thanh cuộn, ô nhập cũng đổi màu
- Thử: DevTools → Rendering → *Emulate prefers-color-scheme: dark*

Nesting gốc — gom kiểu của một khối

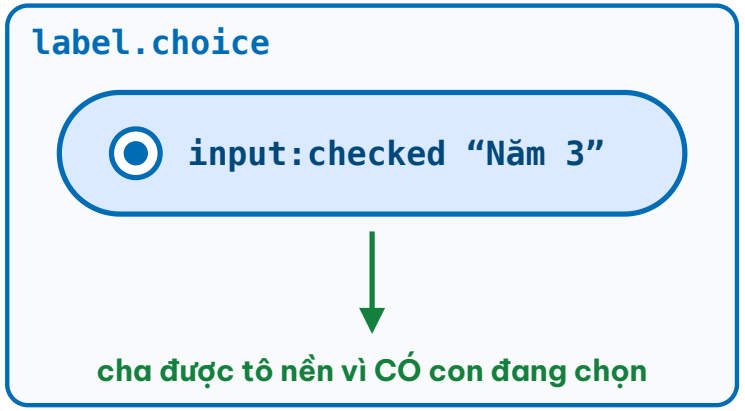
```
.card {
  …
  &:hover {
    translate: 0 -4px;
    box-shadow: var(--shadow-lg);
  }

  &:has(.badge--hot) {
    border-color: var(--color-warning);
    …
  }
}
```

Đoạn mã 2.21. Nesting gốc trong card. Nguồn: `02_Projects/ch02/solution/css/components.css`

- `&` = "chính selector cha" → `&:hover` nghĩa là `.card:hover`
- Lồng được cả `@media` (đã thấy ở 2.5)
- CSS **thuần**, không cần Sass; chạy trên Chrome/Edge/Firefox/Safari từ 2024
- Đừng lồng quá 3 cấp → selector dài, điểm cao, khó ghi đè

2.6 `:has()` — chọn cha theo con



Hình 2.23. Nhãn cha được tô nền vì chứa radio đang chọn (tự vẽ).

```
.choice {
  …
  &:has(input:checked) {
    border-color: var(--color-primary);
    background-color: var(--color-surface-muted);
    font-weight: var(--weight-semibold);
  }
}
```

Đoạn mã 2.22. Radio dạng viên thuốc tô nền bằng `:has()`. Nguồn: `css/components.css` (ch2-T6). Ngoài ra: `.card:has(.badge--hot)` đổi viền card “Sắp hết chỗ” (ch2-T4).

Bấm thử (chạy thật): ◯ Năm 1 ◯ Năm 2 ◯ Năm 3

2.6 `:is()` và `:where()` — viết gọn danh sách

```css
/* :where() có specificity = 0 → lớp sau ghi đè thoải mái */
:where(body, h1, h2, h3, p, ul, ol, dl, dd, figure, fieldset) {
  margin: 0;
}
```

Đoạn mã 2.23. Reset margin bằng `:where()`. Nguồn: `02_Projects/ch02/solution/css/base.css`

```css
.table {
  …
  & :is(th, td) {
    …
  }
}
```

Đoạn mã 2.24. Dùng `:is()` cho ô bảng. Nguồn: `css/components.css`

Bảng 2.7. So sánh `:is()` và `:where()`

	`:IS(A, B)`	`:WHERE(A, B)`
Nghĩa	"là a *hoặc* b"	
Điểm	Cao nhất trong danh sách	Luôn **0-0-0**
Dùng	Kiểu của component	Reset, mặc định

2.6 Transition — chuyển mượt khi rê chuột

```
.card {
  …
  transition:
    translate var(--duration-normal) var(--ease-out),
    box-shadow var(--duration-normal) var(--ease-out);

  &:hover {
    translate: 0 -4px;
    box-shadow: var(--shadow-lg);
  }
}
```

Đoạn mã 2.25. Card nhấc lên mượt khi rê chuột. Nguồn: `02_Projects/ch02/solution/css/components.css`

- **transition**: đổi từ trạng thái A → B một cách mượt (rê chuột, focus)
- Liệt kê *đúng* thuộc tính cần chuyển; ưu tiên `translate`, `opacity`, bóng đổ

Rê chuột vào đây

transition thật trên slide

```
animation: pulse 1.6s ease-in-out infinite;

@keyframes pulse {
  50% {
    opacity: 0.3;
  }
}
```

Đoạn mã 2.26. Chấm nhấp nháy của nhãn "Sắp hết chỗ". Nguồn: `css/components.css` (chấm của `.badge--hot`)

● SẮP HẾT CHỖ ← chạy thật

```
@media (prefers-reduced-motion: reduce) {
  …
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Đoạn mã 2.27. Tắt chuyển động khi người dùng yêu cầu. Nguồn: `css/base.css`

animation + `@keyframes` tự chạy, lặp lại. Có người bị chóng mặt khi màn hình chuyển động → họ bật "Giảm chuyển động" trong hệ điều hành, trang phải tôn trọng. Đây là chỗ **duy nhất** được dùng `!important` (ch2-T2).

2.6 Lỗi hay gặp – CSS hiện đại

✗ Viết `color: #003f87` thẳng trong component	✓ `color: var(--color-primary)` – dark mode tự đúng
✗ Dark mode viết lại `.card`, `.btn`... trong `@media (dark)`	✓ Chỉ đổi giá trị token trong `:root`
✗ `var(--color-primay)` gõ sai tên → màu mất, không báo lỗi	✓ DevTools → Computed; dùng autocomplete của VS Code
✗ Dùng `:invalid` → form đỏ lòm ngay khi vừa mở	✓ `:user-invalid` chỉ báo lỗi sau khi người dùng nhập/rời ô
✗ `transition: all` trên mọi thứ	✓ Liệt kê thuộc tính cần chuyển; tôn trọng reduced-motion

2.6 Thử ngay: một token, cả trang đổi

THỬ NGAY · 5 PHÚT

1. Mở `index.html` (solution) → DevTools → chọn `:root` (thẻ `html`) → sửa `--palette-blue-700` thành `#7c3aed`. Bao nhiêu chỗ đổi màu?
2. Rendering → *prefers-color-scheme: dark*. Chụp màn hình trước/sau.
3. Rendering → *prefers-reduced-motion: reduce*: rê chuột vào card, chấm "Sắp hết chỗ" còn nhấp nháy không?
4. Trên `register.html`: gõ MSSV 9 chữ số rồi bấm Tab. Ô đổi viền đỏ nhờ selector nào?

- Gợi ý: nút, link, tiêu đề card… đổi theo vì đi qua `--color-primary`; câu 3: không nhấp nháy, card không nhảy; câu 4: `&:user-invalid` trong `.field`.

2.7

PHẦN 7 / 7

PHẦN 7

Tổ chức CSS: BEM & Tailwind

- Đặt tên class kiểu BEM
- Utility-first với Tailwind CSS v4
- `@theme` thay cho `tailwind.config.js`

BEM — đặt tên class có hệ thống



Ví dụ đời thường — địa chỉ nhà:

Khu A (block) → căn hộ 12 (element) → căn góc (modifier): đọc tên là biết nó thuộc đâu, khác gì.

Trong EduHub: .card · .card__media · .card__body · .card__title · .btn · .btn--primary · .btn--ghost

Hình 2.24. Giải phẫu tên class BEM (tự vẽ).

Tên class dài hơn nhưng **phẳng** (điểm 0-1-0), không đụng nhau giữa các khối, đọc HTML là đoán được CSS.

```
<article class="card event-card">
  <img class="card__media" src="assets/club-tin-hoc.svg" alt="…">
  <div class="card__body">
    <p class="card__meta">… <span class="badge badge--hot">…</span></p>
    <h3 class="card__title"><a href="event-detail.html">Workshop: …</a></h3>
    …
    <p class="event-card__seats">Còn 12/60 chỗ</p>
  </div>
</article>
```

Đoạn mã 2.28. HTML card sự kiện đặt tên theo BEM. Nguồn: `02_Projects/ch02/solution/events.html` (rút gọn thuộc tính và chữ)

```
.card:has(.badge--hot) .event-card__seats {
  color: var(--color-warning);
}
```

`event-card` là block "mở rộng" của `card`: một thẻ mang cả hai class `card event-card`. Element luôn gắn với block: `card__title`, không phải `card__body__title`.

Utility-first: một hướng khác — Tailwind CSS

CSS thuần + BEM (Ch2)

```
<article class="card event-card">
```

- Đặt tên class theo **ý nghĩa**
- Kiểu viết trong tệp `.css` riêng
- HTML gọn, CSS dài

Utility-first (Tailwind)

```
<article class="group relative flex
  flex-col overflow-hidden rounded-card
  border border-line bg-white shadow-sm
  hover:-translate-y-1 …">
```

- Mỗi class = **một việc nhỏ** (`flex`, `p-6`)
- Gần như không viết tệp CSS

Đoạn mã 2.29. Cùng một thẻ card viết theo hai cách. Nguồn: `02_Projects/ch02/solution/events.html` và `challenge-tailwind/events.html` (rút gọn class)

Ch2 học **CSS thuần** để hiểu gốc rễ. Tailwind thành công cụ chính từ **Ch4** (React).

2.7 Tailwind CSS v4: cấu hình bằng `@theme` trong CSS

```
@import "tailwindcss" source(none);
@source "../events.html";

@theme {
  --color-primary: #003f87;
  --color-primary-light: #7fb2ff;
  …
  --breakpoint-sm: 40rem;
  --breakpoint-lg: 64rem;
}
```

Đoạn mã 2.30. Cấu hình Tailwind v4 bằng `@theme`.
Nguồn: `02_Projects/ch02/solution/challenge-tailwind/src/input.css`

- **Không còn** `tailwind.config.js` (kiểu v3)
- `--color-primary` trong `@theme` → sinh class `bg-primary`, `text-primary`
- Token chép từ `tokens.css` → Ch4 dùng lại
- Tiền tố: `sm:` `lg:` `dark:` `hover:` `has-[…]:`
- Build: `npm run build`

2.7 Lỗi hay gặp — tổ chức CSS

✗ `.card .title`, `.box2`, `.mau-xanh` → ✓ `.card__title`, `.btn--primary`: tên nói lên vai trò

✗ Lồng BEM: `.card__body__title` → ✓ Element thuộc block, không thuộc element: `.card__title`

✗ Tailwind v4 mà đi tạo `tailwind.config.js` theo hướng dẫn cũ → ✓ `@import "tailwindcss";` + `@theme { … }` trong CSS

✗ Nộp cả `node_modules/` lên GitHub → ✓ Có `.gitignore`; người chấm tự `npm install`

2.7 Thử ngay: đặt tên BEM

THỬ NGAY · 4 PHÚT

1. Đặt tên BEM cho hộp FAQ gồm: khối ngoài, từng câu hỏi, phần trả lời, và câu hỏi "đang mở".
2. Card CLB có tên, mô tả, và biến thể "nổi bật" viền vàng: viết 3 tên class.
3. Đọc class Tailwind `sm:sticky sm:top-0`: tương đương CSS nào trong `layout.css`?

- Gợi ý: `.faq` · `.faq__item` · `.faq__answer` · trạng thái mở dùng `[open]` sẵn có (solution: `.faq__item[open]`) hoặc `.faq__item--open`; CLB: `.club-card`, `.club-card__name`, `.club-card--featured`; câu 3 = `@media (width >= 40rem) { position: sticky; inset-block-start: 0 }` của `.site-header`.

Tổng kết Chương 2

- **Cascade**: nguồn → `@layer` → specificity → thứ tự. Dùng class, tránh ID và `!important`
- **Box model** + `box-sizing: border-box`; đơn vị `rem`, `clamp()` cho chữ co giãn
- **Flexbox** một chiều (header, menu, thân card) · **Grid** hai chiều (lưới card, trang chi tiết)
- `repeat(auto-fit, minmax(min(16rem, 100%), 1fr))` — lưới tự đổi cột, không media query
- **Mobile-first**: CSS mặc định cho điện thoại, 2 mốc `40rem` / `64rem`; không cuộn ngang ở 375px
- **Design token** bằng custom properties → dark mode chỉ đổi token; `:has()`, nesting, `:user-invalid`
- Tôn trọng người dùng: focus rõ, vùng chạm ≥ 44px, `prefers-reduced-motion`
- Đặt tên **BEM**; Tailwind v4 (`@theme`) là hướng utility-first — dùng chính từ Ch4

Câu hỏi tự kiểm tra

✔ CHECKPOINT — CHƯƠNG 2

1. `.nav a` và `a:hover` cùng đổi màu link. Điểm specificity mỗi bên? Nếu bằng nhau thì sao?
2. Hộp `width: 300px; padding: 16px; border: 2px` rộng bao nhiêu khi `content-box`? `border-box`?
3. Khi nào chọn Flexbox, khi nào chọn Grid? Cho một ví dụ mỗi loại trong EduHub.
4. Vì sao CSS mặc định (ngoài `@media`) nên viết cho điện thoại?
5. Làm dark mode mà không sửa component: token đặt ở đâu, đổi ở khối nào?

NHÌN LẠI Quay lại 3 câu ôn tập đầu buổi — câu 3 giờ bạn trả lời thế nào?

Bảng 2.8. Nhiệm vụ bài tập Chương 2 (khớp `solution/TASKS.md`)

MÃ	CẤP	VIỆC	ĐIỂM
`ch2-T1`	Warm-up	Design token + dark mode (`tokens.css`)	15
`ch2-T2`	Warm-up	`@layer` + `@import`; base: box-sizing, focus, reduced-motion	15
`ch2-T3`	Applied	Khung mobile-first: header Flexbox, hero 1→2 cột, nút, skip link	20
`ch2-T4`	Applied	Lưới card `auto-fit`, card BEM + nesting, `:has()`, FAQ	20
`ch2-T5`	Applied	Trang chi tiết `grid-template-areas`, bảng cuộn trong khung	15
`ch2-T6`	Applied	Form 2 cột, radio `:has(input:checked)`, `:user-invalid`	15
`ch2-T7`	Challenge	Trang sự kiện bằng Tailwind CSS v4 + `@theme`	+10

Starter: `02_Projects/ch02/starter/` — HTML có sẵn, chỉ viết CSS theo `TODO(ch2-Tk)`. Repo nộp: `EduHubWeb-Ch02-<MSSV>`. Không float, không `!important` (trừ reduced-motion), không ID selector.

Xem trước Chương 3 — JavaScript hiện đại & DOM

Chương 3 học gì?

- JavaScript ES2025: biến, hàm, mảng, đối tượng
- DOM: tìm phần tử, sự kiện click, submit
- `fetch` + `async/await` gọi mock API
- Card sự kiện được **sinh từ dữ liệu API** thay cho HTML tĩnh
- Tìm kiếm, lọc, đăng ký sự kiện thật

CSS Ch2 giữ nguyên — JS chỉ thêm/bớt class.

Cần chuẩn bị

- Xong ch2-T1…T6, push lên GitHub, bật Pages
- Kiểm 375 / 768 / 1280px: không cuộn ngang
- `npx -y html-validate@latest "*.html"` → 0 lỗi
- Cài **Node.js 24 LTS** (bắt buộc từ Ch3), chạy `node -v`
- Đọc MDN "JavaScript first steps"